

Ejercicio 1

En este ejercicio se trabaja con una clave de **128 bits**, la cual no se obtiene a partir de un algoritmo criptográfico en sí, sino mediante una operación **XOR**. El objetivo principal no es cifrar información, sino **disociar** la clave final, evitando que esta esté escrita directamente en el código.

La idea es generar la clave final en memoria combinando dos partes:

- una **clave fija**, que es común a todo el sistema
- una **clave dinámica**, que cambia según el entorno (desarrollo o producción)

De esta forma, aunque la clave fija sea siempre la misma, la clave final será distinta dependiendo del entorno.

Entorno de Desarrollo

Para el entorno de **desarrollo** se utilizan los siguientes valores:

- **Clave Fija (general):**
B1EF2ACFE2BAEEFF
- **Clave dinámica:**
91BA13BA21AABB12

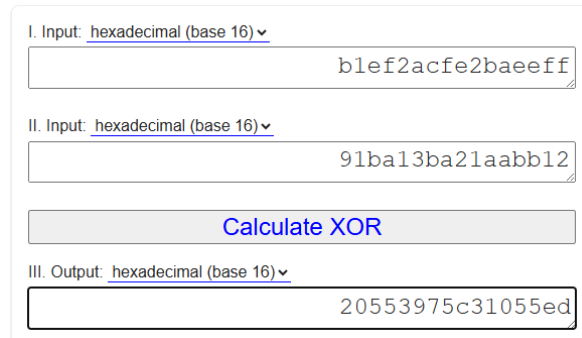
Ambos valores se combinan mediante una operación **XOR** y como resultado se obtiene una **clave final**:

- **Clave final generada en memoria:**
20553975c31055ed

Para este calculo podemos usar algo como “**XOR Calculator**”:

XOR Calculator

Thanks for using the calculator. [View help page](#).



I. Input: [hexadecimal \(base 16\)](#) ▼
b1ef2acfe2baeef

II. Input: [hexadecimal \(base 16\)](#) ▼
91ba13ba21aabb12

[Calculate XOR](#)

III. Output: [hexadecimal \(base 16\)](#) ▼
20553975c31055ed

Esta clave final no se almacena ni se muestra normalmente, sino que existe únicamente en memoria durante el funcionamiento.

Entorno de Producción

En el entorno de **producción** se utiliza la misma **clave general**, pero cambia la clave dinámica, la cual se encuentra almacenada en el **keystore**:

- **Clave fija (general):**
B1EF2ACFE2BAEEFF
- **Clave dinámica (keystore):**
B98A15BA31AEBB3F

Al aplicar nuevamente la operación XOR, se obtiene una clave final distinta:

- **Clave final generada en memoria:**
08653f75d31455c0

Ejercicio 2

Parte 1

En este ejercicio trabajamos con **AES**, que es un algoritmo de **cifrado simétrico en bloque**. Aunque el sistema utilice una clave de 256 **bits**, el tamaño de bloque de **AES** es siempre de **128 bits**, por lo que todo el cifrado se realiza sobre bloques de ese tamaño.

Se nos proporciona un mensaje **cifrado** en **Base64**, junto con la clave almacenada en el **keystore**, que se encuentra en formato **hexadecimal**:

- **Mensaje cifrado (Base64):**

TQ9SOMKc6aFS9SlxhfK9wT18UXpPCd505Xf5J/5nLI7Of/o0QKIWXg3nu1RRz4QWElezdrLAD5LO4USt3aB/i50nvvJbBiG+le1ZhpR84oI=

- **Clave (keystore hexadecimal):**

A2CFF885901A5449E9C448BA5B948A8C4EE377152B3F1ACFA0148FB3A426DB72

También se nos indica un IV compuesto únicamente por ceros: **00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00**

Esto se debe a que la consigna especifica que el **IV** estará expresado en hexadecimal correspondiente a ceros binarios. Como el **IV** debe tener el mismo tamaño que el **bloque**, y en **AES** el bloque es de **128 bits**, esto equivale a **16 bytes**. Cada **byte** en **hexadecimal** se representa con dos caracteres, por lo que el **IV** termina siendo una cadena de **32 ceros**.

Para continuar con el proceso, el mensaje cifrado se pasa de **Base64 a hexadecimal**, quedando de la siguiente forma:

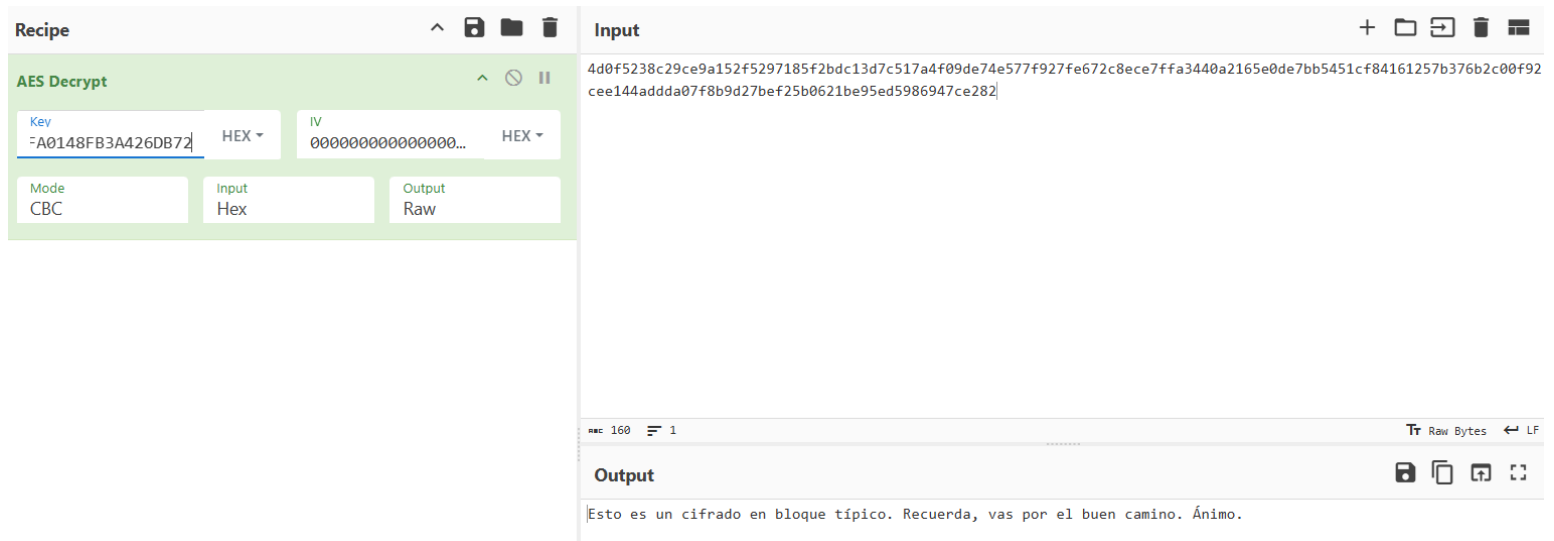
- **Mensaje cifrado (hexadecimal):**

4d0f5238c29ce9a152f5297185f2bdc13d7c517a4f09de74e577f927fe672c8ece7ffa3440a2165e0de7bb5451cf84161257b376b2c00f92cee144addda07f8b9d27bef25b0621be95ed5986947ce282

Con todos los valores necesarios, se procede a realizar el descifrado utilizando **AES** en modo **CBC** con **padding PKCS7**, ya sea mediante CyberChef o Python. En este caso se hizo en **cyberchef**

El resultado del descifrado es el siguiente mensaje en texto plano:

“Esto es un cifrado en bloque típico. Recuerda, vas por el buen camino. Ánimo.”



Ejercicio 3

En este ejercicio el objetivo es **cifrar** el texto:

“KeepCoding te enseña a codificar y a cifrar”

utilizando una clave almacenada en el **keystore** bajo el identificador **“cifrado-sim-chacha-256”**, junto con un **nonce** dado, y migrar el esquema de cifrado desde **ChaCha20** a **ChaCha20-Poly1305**.

ChaCha20, en su forma básica, es únicamente un **algoritmo de cifrado**, lo que significa que su función principal es garantizar la **confidencialidad** del mensaje, es decir, que el contenido no pueda ser leído por terceros. Sin embargo, ChaCha20 por sí solo **NO** permite detectar si el mensaje fue **modificado** durante la transmisión.

Esto es similar a lo que ocurre con algoritmos como **AES** en modo **CBC**: el mensaje queda cifrado, pero si un atacante modifica el ciphertext, el receptor no tiene forma directa de saberlo y el descifrado puede generar información incorrecta sin avisar.

En sistemas actuales, cifrar solamente **no es suficiente**. Además de confidencialidad, también se necesita garantizar:

- **Integridad**, para saber si los datos fueron alterados.

- **Autenticidad**, para tener la certeza de que el mensaje proviene de quien dice enviarlo.

En el caso de **AES**, este problema se soluciona utilizando modos autenticados como **AES-GCM**, que combinan cifrado con un mecanismo de autenticación mediante un **tag**.

Para **ChaCha20**, la solución equivalente es **ChaCha20-Poly1305**. En este formato, ChaCha20 se encarga del cifrado del mensaje, mientras que **Poly1305** genera un **tag de autenticación (MAC)** calculado sobre el ciphertext. De esta manera, cualquier modificación del mensaje, incluso de un solo bit, produce un **tag** inválido y el descifrado **falla**, dejando en claro que el mensaje fue alterado.

En conclusión, migrar de ChaCha20 a **ChaCha20-Poly1305** permite pasar de un esquema que solo garantiza confidencialidad a uno de **cifrado autenticado (AEAD)**, donde se obtiene tanto el mensaje cifrado como un tag que asegura la integridad y autenticidad de los datos.

Ejercicio 4

En este ejercicio se nos da un JWT junto con la clave secreta “Con KeepCoding aprendemos”. Un **JWT** está formado por **tres partes** separadas por puntos. El **Header**, el **Payload** y la **firma**, estando solo las dos primeras codificadas en Base64.

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9 -
eyJ1c3VhcmIvIjoiRG9uIFBlcGl0byBk
ZSBsb3MgcGFsb3RlcyIsInJvbCI6ImIzQWRtaW4iLCJpYXQiOiJlE2Njc5MzM1M
zN9 -krkBkzCBQ5WZ8JnZHuRvmnAZdg4ZMeRNv2CIAODIHRI

(Solo una forma visual de entenderlo **HEADER** - **PAYLOAD** - **FIRMA**)

Last build: 4 months ago - version 1.0 is here! Read about the new features here

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars ☐ Strict mode

Input

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c3VhcmIvIjoiriR69uIFB1cG10byBkZSBSb3MgcGFsb3RlcyIsInJvbCI6Im1zTm9ybWFsIiwiaWFnZW0iOiJ0eXNjY3OTMzNTMzFQ.gfhw0dXp6oixMLXXRP97W4TDTrv0y7B5VjD0U8ixrE

Output

```
{ "typ": "JWT", "alg": "HS256" } { "usuario": "Don Pepito de los palotes", "rol": "isNormal", "iat": 1667933533 } ES US • CS 51 508 zq, L-ufi?p0a00*y21 es X* =pxi0, k
```

La tercera parte corresponde a la firma, la cual no es legible y sirve únicamente para verificar que el contenido del token no haya sido modificado.

Como me quedo la duda de querer comprobar con la firma, busque alternativas para hacer esto, y me tope con que se puede utilizar **JWT Debugger (jwt.io)**. Al ingresar el **token** original junto con la clave secreta, la firma se valida correctamente. En cambio, al modificar el payload, la

firma deja de ser válida, confirmando que sin la clave no es posible generar un JWT correcto.

The screenshot shows the JWT Debugger interface. On the left, the 'ENCODED VALUE' section displays a 'Valid JWT' with a 'Signature Verified' status. The token string is: `eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c3VhcmliIjoiaW9uIFB1c256byBkZSsb3MgcGFsb3R1cyIsIn3VbCI6Im1zTm9ybWFsIiwiaWF0IjoxNjY3OTMzNTMzZQ.gfhw0dDxp6oixMLXXRP97W4TDTrv0y785Vjd0U8ixrE`. On the right, the 'DECODED HEADER' section shows the JSON header: `{ "typ": "JWT", "alg": "HS256" }`. Below it, the 'DECODED PAYLOAD' section shows the JSON payload: `{ "usuario": "Don Pepito de los palotes", "rol": "isNormal", "iat": 1667933533 }`. At the bottom, the 'JWT SIGNATURE VERIFICATION (OPTIONAL)' section shows a 'Valid secret' and the text 'Con KeepCoding aprendemos'.

Ejercicio 5

MENSAJE: “En KeepCoding aprendemos
cómo protegernos con criptografía”

HASH -SHA3 KECCAK = DEL MENSAJE:

`bced1be95fbd85d2ffcce9c85434d79aa26f24ce82fbd4439517ea3f072d56fe`

Pregunta 1: ¿Qué tipo de SHA3 hemos generado?

Respuesta: Hemos generado un **SHA3-256**, ya que contamos con que nuestro hash en este caso **tiene 64 caracteres hexadecimales**. Y sabemos que **2 hexadecimales** es igual a **un byte**, entonces si dividimos los 64 hexadecimales en 2, **nos da 32 bytes**, por eso usamos el **SHA3 -256**, otro calculo que serviría para confirmar seria multiplicar esos 32 bytes x 8 (1 bytes es igual a 8 bits) y nos dará 256 bits. En conclusión SHA3-256 bits

Pregunta 2: Y si hacemos un SHA2 con el resultado :

4cec5a9f85dcc5c4c6ccb603d124cf1cdc6dfe836459551a1044f4f2908aa5d63739506f

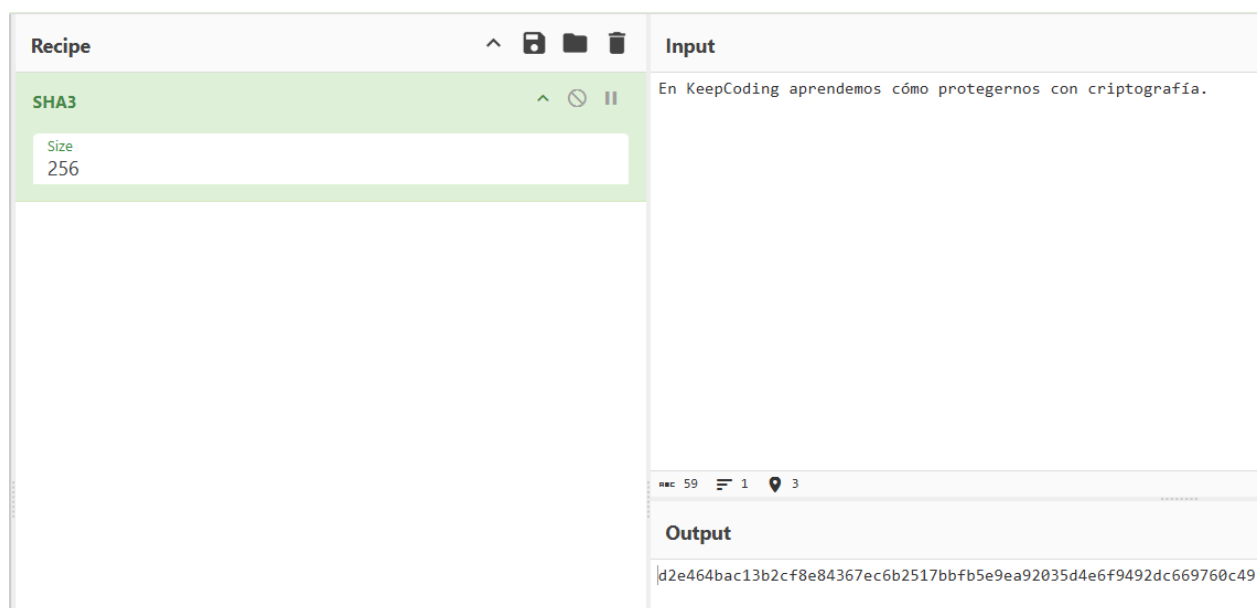
6468833d77c07cfd69c488823b8d858283f1d05877120e8c5351c833

Respuesta: Al igual que en la pregunta anterior, contamos la cantidad de caracteres hexadecimales, en este caso son **128 hex**, que si lo dividimos en 2 porque como dije antes 2 hex son 1 byte, nos dan 64 bytes y esto por regla queda emparejado con el sistema de **SHA2 - 512 bit**, porque si multiplicamos **64 bytes x 8 bits** nos dan 512 bits

Pregunta 3: Genera ahora un **SHA3** Keccak de **256 bits** con el siguiente texto: “En KeepCoding aprendemos cómo protegernos con criptografía.”
¿Qué propiedad destacarías del **hash**, atendiendo a los resultados anteriores?

Respuesta= Al generar un SHA3 Keccak del mensaje “En KeepCoding aprendemos cómo protegernos con criptografía” me arrojó el siguiente **hash**:

d2e464bac13b2cf8e84367ec6b2517bbfb5e9ea92035d4e6f9492dc669760c49



Lo que podría destacar del **hashing** en general y del punto o propiedad en común entre los resultados anteriores es que no importa la longitud del mensaje, y dependiendo de que tipo de **SHA** en este caso utilices, siempre recibiremos un **hash** de la longitud hexadecimal asignada al valor o el tipo de **hashing** que estemos utilizando, aquí como el parámetro a usar era un **SHA3-de 256 bits** nos dio un hash en hexadecimal de **64 caracteres**

Otro punto importante a tener en cuenta, es que en el caso de esta pregunta y viendo el mensaje a hashear este tenía un “•” (punto final) el cual no sabía si iba, y al ver cuál era el cambio entre ambas opciones agregando este punto, es que un mínimo **bit** o cambio y el hashing aunque manteniendo su longitud, es completamente distinto, por lo que podemos confirmar que la analogía que se suele hacer de que el **hash** es como si fuese una **huella digital** del mensaje es completamente correcta

Ejercicio 6

Pregunta: calcula el hmac-256 (usando la clave contenida en el Keystore) del siguiente texto: Siempre existe más de una forma de hacerlo, y más de una solución válida

CLAVE EN KEYSTORE :

A212A51C997E14B4DF08D55967641B0677CA31E049E672A4B06861AA4D5826EB

Respuesta:

234971de1a8fe04bba2e1dfa8483b9e2653a3b76e2aa2a37ee4aa0f9787c210b

Utilizando cyberchef en **input** pegamos el mensaje, luego importamos **HMAC** a la receta y dentro de los parámetros ingresamos la **clave** proporcionada en por el keystore de la práctica y en sus parámetros de hashing ingresamos que era el **algoritmo SHA256**

Ejercicio 7

Empresa de desarrollo: Aplicación Web

Requiere ingreso con: Login → Passwords

Objetivo: Setear un sistema de almacenamiento de hash para las contraseñas

Pregunta A: ¿Por qué crees que **SHA-1** es una mala opción?

Respuesta A: Podemos considerar que SHA-1 es una mala opción ya que este es muy fácil de romper a través de la búsqueda intencionada de colisiones, es decir dos mensajes con el mismo hash (misma identidad), esta desactualizado y es obsoleto al día de hoy 2025

Pregunta B: Se propone el uso de un SHA-256 (SHA-2), como reemplazo a la primera propuesta. ¿Cómo se podría mejorar o fortalecer en una primera instancia?

Respuesta B: La mejor forma de mejorar el uso de SHA-256 (SHA-2) en primera instancia sería agregarle un **SALT**. Un valor único y aleatorio que se mete básicamente antes de hashear la contraseña. Con esto logramos que, aunque dos usuarios tengan exactamente la misma contraseña, el hash que genera cada una quede totalmente distinto evitando complicaciones y ataques.

Pregunta C: El responsable ha quedado conforme con la propuesta pero... ¿Hay algún margen de mejora extra luego del **SALT**?

Respuesta C: Además del SALT, en una segunda instancia de mejora para SHA-256 podemos agregar un **PEPPER**. Es un valor secreto que también se combina con la contraseña al momento de hashearla, pero a diferencia del SALT este no se guarda en la base de datos. El **PEPPER** suma una capa extra de protección si la base es robada, porque sin ese valor secreto no se puede reproducir el hash. Y a diferencia del SALT, que es único para cada usuario, el **PEPPER** es global (hay uno solo para todos los usuarios o para un grupo), lo que permite mantenerlo fuera de la base de datos y usarlo como secreto adicional extra

Ejercicio 8

En este ejercicio tenemos una **API REST** (Comunicación entre dos sistemas). La cual se divide en dos partes o momentos. **REQUEST** y **RESPONSE**.

La **Request** podríamos decir que sería lo que el cliente o usuario solicita al servidor cuando ingresa al sistema, ingresando ciertos datos, mientras que la **Response** sería lo que devuelve el servidor al cliente (por ejemplo un usuario con App bancaria y lo que devuelve el servidor bancario de nuevamente al cliente a través de la app)

En la **API** y para ambas etapas ya mencionadas tenemos ciertos campos que necesitan:

- **Confidencialidad** → que nadie pueda leerlos. (Privacidad de un extremo a otro)
- **Integridad** → que nadie pueda modificarlos sin que nos demos cuenta.
- **Autenticación** → Implícita en este caso al usar clave secreta compartida

En este caso particular la consigna nos indica que debemos asumir que el sistema End to End **NO** usa **TLS**. ¿Qué significa esto?

Sabiendo que TLS, es lo que normalmente protege a **HTTPS** y a las conexiones seguras del navegador, debemos entender que no podemos confiar en el canal, por lo que tenemos que proteger el contenido que se envía a través de él a toda costa.

Teniendo esto en claro, ahora necesitamos de elegir un mecanismo de cifrado con algoritmo que incluya las características que mencionamos anteriormente y estos podrían ser:

- Algoritmo **AES** con modo de operación **GCM** (Cifrado Simétrico en bloque)
- **ChaCha20-Poly1305** (Cifrado Simétrico por flujo)

Ambas opciones cuentan con la propiedad principal de ser **AEAD**, Cifrado autenticado con datos asociados; Esto significa que cuenta con *Integridad*, *Confidencialidad* y *Autenticación* del mensaje

RESPUESTA FINAL: Si bien podemos afirmar que ambas opciones son excelentes para este ejercicio de caso real, y que ninguno es mejor que el otro, por una simple cuestión de estándar histórico ya que se usa más en el ámbito bancario, usaríamos **AES GCM**, ya que cuenta con todo lo que necesitamos

Ejercicio 9

Objetivo: Calcular el KCV de la siguiente clave AES:

A2CFF885901A5449E9C448BA5B948A8C4EE377152B3F1ACFA0148FB3A426DB72

Debemos hacerlo tanto en **SHA-256**, como a través de **AES** (Solo son dos formas distintas de hacerlo)

- 1- Sabemos que en el caso del KCV (SHA-256), se corresponde con los 3 primeros bytes del SHA-256, entonces al calcular el SHA-256, de la cadena hexadecimal dada nos da:

2ecddbc4a7156a631f169f1dcc2c4174d545d76a826d015882b0b4647837e4d

Recipe

SHA2

Size
256

Rounds
64

Input

A2CFF885901A5449E9C448BA5B948A8C4EE377152B3F1ACFA0148FB3A426DB72

64

1

Output

2ecddbc4a7156a631f169f1dcc2c4174d545d76a826d015882b0b4647837e4d

El **KCV** de este modelo en particular correspondía a los **3** primeros bytes de este **SHA-256**, y como sabemos que **1 BYTE** es igual a **2** caracteres hexadecimales juntos, nuestro **KCV** sería el siguiente: **2E CD DB**

- 2- Sabemos que en el caso del **KCV (AES)**, se corresponde a un texto cifrado con un tamaño de bloque de **16 bytes**, en el cual el mensaje o texto plano es una cadena de **32 ceros** binarios ("00"), con un **IV** también de **32 ceros**.

Con estos dos datos y sumado obviamente a la clave AES que se nos proporcionó anteriormente, solo nos queda seleccionar el modo de operación el cual por regla estándar y ya que usa **IV**, elegiremos **CBC**. Entonces:

Texto Plano:

00000000000000000000000000000000

IV:

00000000000000000000000000000000

Clave:

A2CFF885901A5449E9C448BA5B948A8C4EE377152B3F1ACFA0148
FB3A426DB72

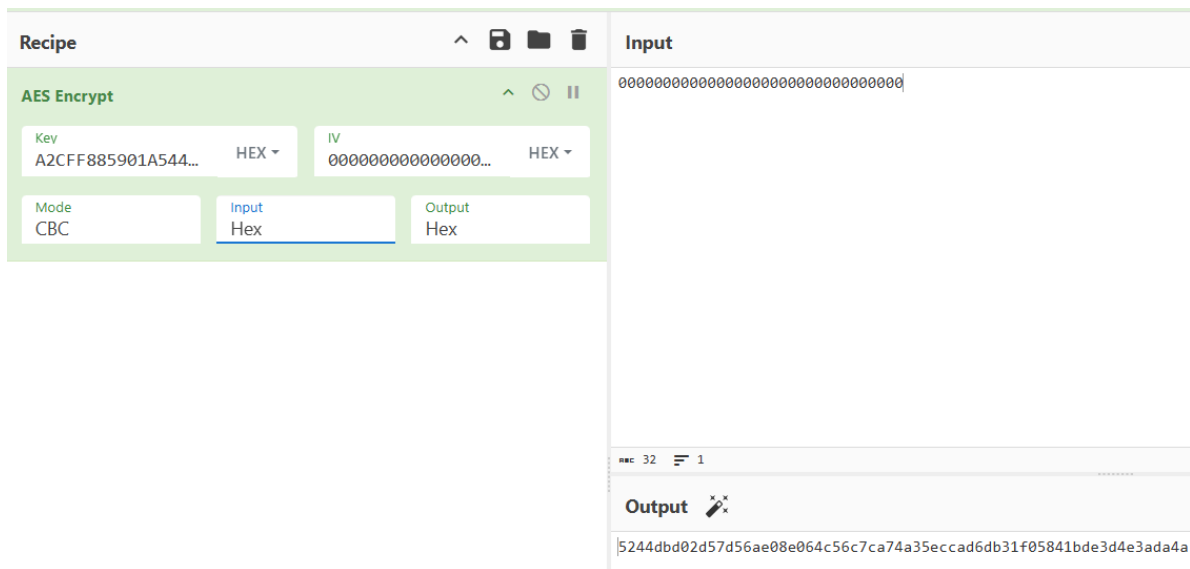
Modo de Operación:

CBC

Una vez tenemos todos estos datos Ciframos y obtenemos la siguiente cadena hexadecimal:

5244dbd02d57d56ae08e064c56c7ca74a35eccad6db31f05841bde3d4e
3ada4a

Sobre la cual si aplicamos un recorte de los 3 primeros bytes como en el primer caso el resultado sería: **52 44 DB**

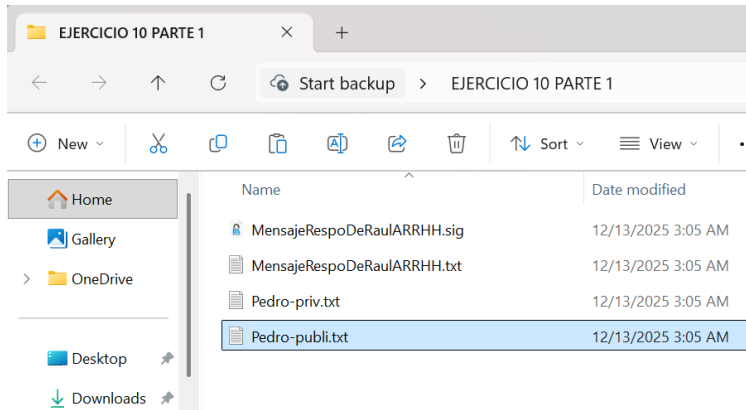


Ejercicio 10

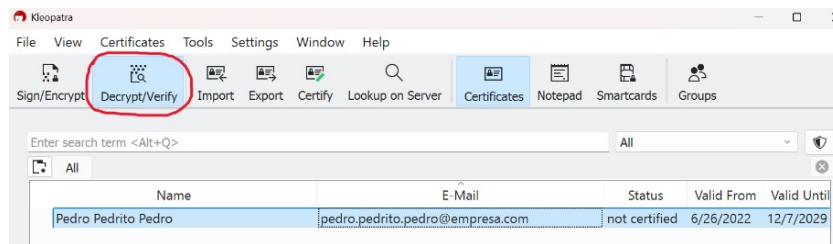
Objetivo: Realizar actividades de Criptografía Asimétrica como personal de **RRHH**. En primera instancia debemos verificar una **firma digital** con **clave pública**. Y en segunda instancia firmar un mensaje con **clave privada**. En ningún caso necesitamos cifrar el mensaje ya que para este contexto solo necesitamos de integridad y autenticidad, y no tanto la confidencialidad por lo que solo realizaremos una Autenticación de firma y luego enviar un mensaje en respuesta con una firma aparte

1. Para esta primer etapa **Pedro**, responsable de **Raúl** envía un mensaje a **RRHH**, donde solicita un aumento de un 20% para Raúl. Como personal de RRHH, hemos recibido un mensaje en archivo **.TXT** acompañado de otro archivo con el mismo nombre, pero con formato **.SIG/GPG** ; Con estos dos mas la clave Publica, que también tenemos, ya podemos autenticar el mensaje.

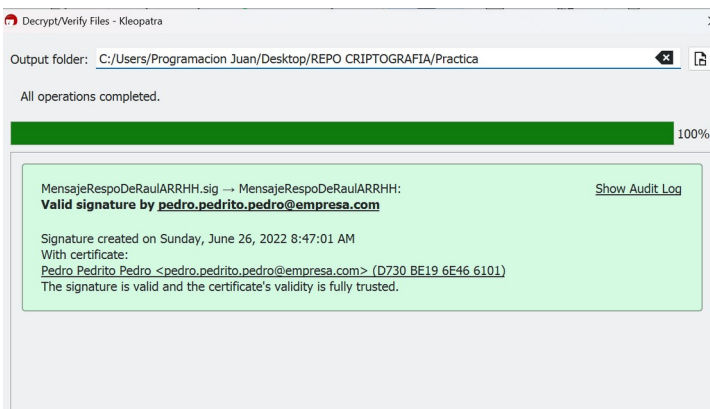
Abrimos **Kleopatra**, u otro programa que funcione como interfaz gráfica para gestión de clave y operaciones **PGP**. Con los siguientes archivos y SIN abrirlos manualmente, abrimos Kleopatra y damos en **import** o Arrastramos primero la **clave publica**



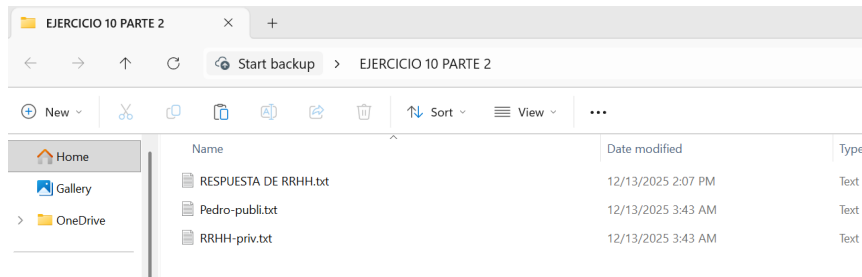
Una vez importada la clave publica nos queda de la siguiente manera ya agregada en nuestro apartado de **Certificados**. Seleccionamos la misma en la lista y le damos a **Decrypt/Verify**.



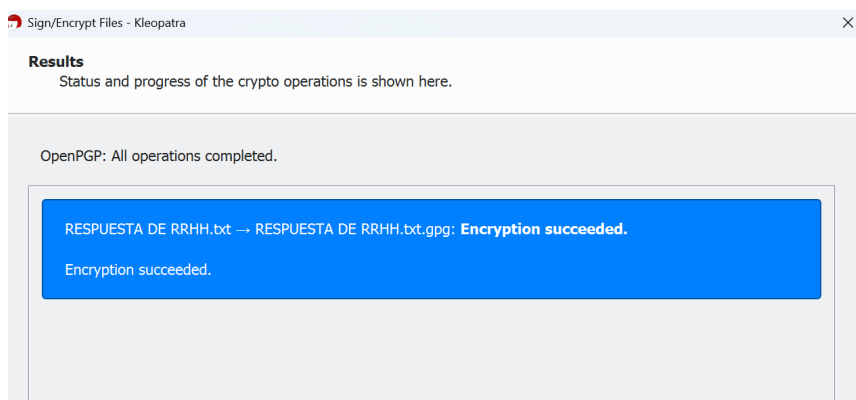
Nos pedirá que importemos algo, ahí seleccionaremos el archivo **.SIG/GPG** que está en la misma carpeta que el **.TXT** del mensaje y se realizara la **Autenticación**



2. En la segunda instancia de la consigna como **personal de RRHH** necesitamos enviar una respuesta y hacer una **Firma Digital** de ella, pero en este caso como se nos pide que cifremos el mensaje, ahora si tenemos que agregar esa capa extra de **confidencialidad** por lo que cifraremos el mensaje. Para esto necesitaremos el mensaje de respuesta en archivo **.TXT**, **clave privada de RRHH** para firmar y la **clave publica de Pedro** para cifrar.

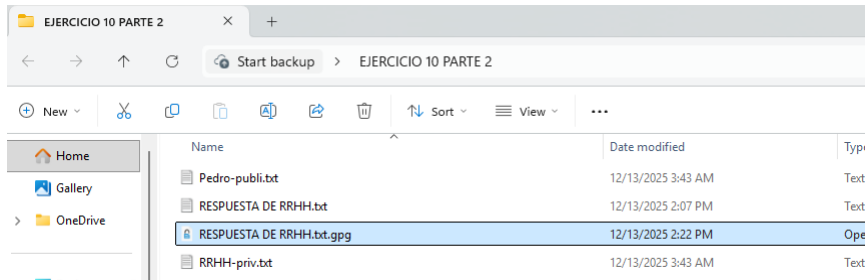


Lo siguiente que haremos será abrir Kleopatra nuevamente, importaremos nuestra **clave privada de RRHH** a la lista de certificados, luego iremos **Sign/Encrypt**, seleccionaremos el archivo **.TXT** de la respuesta y se abrirá una nueva ventana. Abriremos en un apartado interno la opción de **Encriptar para Otros** y ahí seleccionaremos la **clave publica de pedro**

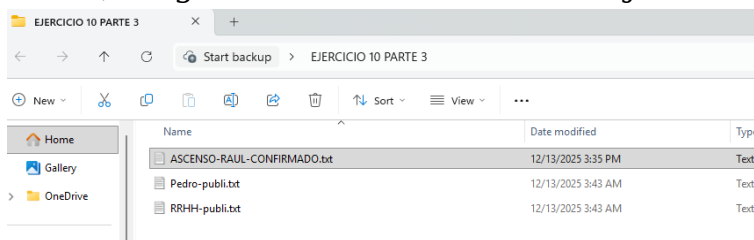


Una vez con esto, tendremos nuestro archivo **.GPG** de la respuesta, es decir la respuesta cifrada y con **Firma digital**

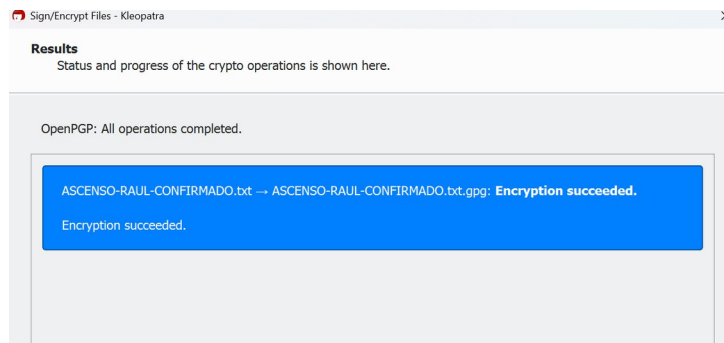
*(Aunque la firma digital y el cifrado son operaciones criptográficas independientes, el sistema PGP permite encadenarlas en un único proceso, generando un fichero **.GPG** que contiene el mensaje firmado y cifrado)*



3. Por último, tenemos un mensaje que actúa como registro o acuerdo de ambas partes donde se consolida que el ascenso de Raúl se va a realizar. Al cifrarlo para ambos, se asegura que tanto RRHH como Pedro, tengan acceso al mismo mensaje cuando sea necesario



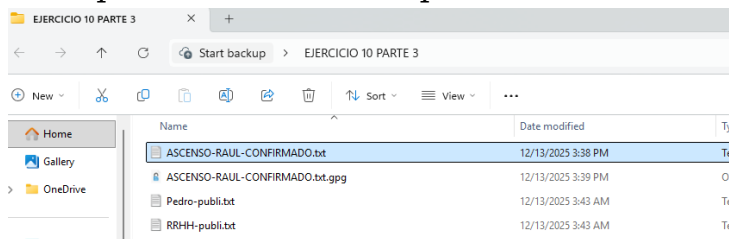
Aquí haremos el mismo procedimiento anterior, nada más que solo importando las claves publicas de RRHH y de Pedro. Luego vamos a la opción de Encrypt, seleccionamos el mensaje nuevo que habla del



acuerdo, destildamos la opción de Firma Digital (ya que solo queremos cifrar), seleccionamos solo encriptar y elegimos ambas claves publicas para que nos quede en solo un archivo **.GPG**

Ejercicio 11

Esta consigna nos pone en el contexto de que formamos parte de una compañía que contrata a otra que brinda un servicio de almacenamiento de



información de videollamadas. Cada una de estas videollamadas tiene su propia **clave simétrica** y utiliza **RSA-OAEP** con **SHA-256** para su cifrado.

Una vez dicho esto, se nos proporciona un texto cifrado correspondiente a una videollamada particular, el cual se encuentra cifrado con **RSA-OAEP** utilizando **SHA-256** como algoritmo interno. Este cifrado fue realizado con nuestra **clave pública** ("**clave-rsa-oaep-publ.pem**"), ubicada en el repositorio de la práctica.

Primero se **descifra** la clave utilizando la **clave privada** ("**clave-rsa-oaep-priv.pem**"), recuperando así la clave simétrica original. Luego se vuelve a cifrar esa misma clave con el mismo algoritmo y se observa que el resultado

es distinto, lo cual es normal, ya que RSA-OAEP introduce **aleatoriedad** para que el cifrado sea más **seguro**.

```
EJERCICIO_11.py
C: > Users > Programacion Juan > Desktop > EJERCICIO 11 > EJERCICIO_11.py > ...

1
2 # EJERCICIO 11 PRACTICA FINAL - CRIPTOGRAFIA
3
4 import os
5 from Crypto.PublicKey import RSA
6 from Crypto.Cipher import PKCS1_OAEP
7 from Crypto.Hash import SHA256
8
9 #ASIGNAMOS UBICACION DE CARPETA DE ESTE SCRIPT y ARCHIVOS.PEM
10 BASE_DIR = os.path.dirname(__file__)
11
12 #INGRESAMOS DATOS DE ENTRADA
13 ciphertext_hex = "b72e6fd48155f565dd2684df3ffa8746d649b11f0ed4637fc4c99d18283b32e1709b30c96b4a8a20d5dbc639e9d83a53681e6d96f76a0e4c279f0dffa76a329d04e3d3d"
14 datos_cifrados = bytes.fromhex(ciphertext_hex)
15
16 clave_privada = RSA.import_key(open(os.path.join(BASE_DIR, "clave-rsa-oaep-priv.pem")).read()) # ASI IMPORTAMOS LAS CLAVES .PEM SI ESTAN
17 clave_publica = RSA.import_key(open(os.path.join(BASE_DIR, "clave-rsa-oaep-publ.pem")).read()) # EN LA MISMA CARPETA QUE ESTE ARCHIVO
18
19 #DESCIFRAMOS
20 mensaje_descifrado = PKCS1_OAEP.new(clave_privada, SHA256).decrypt(datos_cifrados)
21
22 #CIFRAMOS NUEVAMENTE
23 nuevo_cifrado= PKCS1_OAEP.new(clave_publica, SHA256).encrypt(mensaje_descifrado)
24
25 print(mensaje_descifrado.hex())
26 print("El nuevo cifrado es:", nuevo_cifrado.hex())
27
28 print("El cifrado cambia porque RSA-OAEP usa aleatoriedad para mayor seguridad.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> "C:\Users\Programacion Juan\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:/Users/Programacion Juan/Desktop/EJERCICIO 11/EJERCICIO_11.py"
e2cff885901a5449e9c448ba5b948a8c4ee377152b3f1acfa0148fb3a426db72
El nuevo cifrado es: 4f5a498807966abca15e6c53139e5f47a5e9cc94de7cfd14c32e57baf01068234908f92331fbfbc75898a6bdd87aff4e7e3611937198cb7a539c6f36f9b0142b0964079fb1beeb640955e9d11c10937fd5c7e86c8a2ba75ff9192db08ca2d46613b7bbed81b34667eb749eaafd574e5b7658cad90e9adb37db281ad803b5288867185add9fe2dfcb78d30a6c14587f59c653523e39af727411bea11e7d62f98e16dbdf93fa0d8a72700b77facbc6074698beaef1e4aa0ae109cda378f2fa64be149ec4237146e68c5337983981e91bcf0b45ea6ab58ffb9b496fbc0f55f1a70a8c8d1b6537a1ca9eda694438e272931e23dc8b049d472f3584271874e8796352
El cifrado cambia porque RSA-OAEP usa aleatoriedad para mayor seguridad.
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code>
```

Explicando un poco lo que se puede ver en este script hecho en Python, noté que al pegar directamente el contenido de las claves **.pem** dentro del código, el script quedaba demasiado largo y poco práctico. Por ese motivo decidí investigar por mi cuenta si existía una forma más simple de trabajar con las claves, y encontré que era posible importarlas directamente desde archivos externos indicando su ruta y ubicación.

Para simplificar aún más el proceso, en las líneas 9-10, se define que la carpeta base donde se encuentra ubicado el script sea la misma en la que se encuentran las claves. De esta manera, el script busca automáticamente los archivos **.pem** en esa ubicación, por lo que decidí colocar las claves en la misma carpeta que el archivo Python.

 EJERCICIO_11.py	12/14/2025 3:17 AM	Python Source File	2 KB
 clave-rsa-oaep-publ.pem	12/13/2025 3:43 AM	CMS (S/MIME) File	1 KB
 clave-rsa-oaep-priv.pem	12/13/2025 3:43 AM	CMS (S/MIME) File	2 KB

Luego procedimos a ingresar los “datos de entrada”, **ciphertext_hex** (mensaje cifrado en formato hexadecimal, línea 13). Dado que Python trabaja con datos binarios representados como bytes ,fue necesario convertir después la cadena hexadecimal del mensaje a bytes (línea 14).

A continuación, se importan las **claves pública** y **privada** de la forma mencionada anteriormente (líneas 16 y 17).

Por último, se procede a **descifrar** la clave y a **cifrarla nuevamente** para comprobar que el nuevo cifrado es diferente en cada ejecución del script. Para dejar asentado este comportamiento, en la línea 28 se agrega un print que aclara lo siguiente: “**El cifrado cambia porque RSA-OAEP usa aleatoriedad para mayor seguridad**”.

Resultado en Terminal:

```
e2cff885901a5449e9c448ba5b948a8c4ee377152b3f1acfa0148fb3a426db72
```

El nuevo cifrado es:

```
0d877841cf14fc1225e25d0153dda7c85aec00b48c34344decbeea2a88bb446
d3a9f9950dde5faf6946740ab8965bbb062550284fcddcf99bfb7a189364b06
79157b35c62fd393dda24900731c96d608f254d748efff69a171fbafc965d36df
c1fdb6e9e4a8ff70067b54acde48e38047afe78d8b8a522307e0a837584b6b0
b5623d1867f24b25715df1dc56c6845e14d9881a2543a7848b87dfdb6f0ea9b
a2e3cadfb6966ef3be1d23f33372bbc43e208efd9551840fad32b9edf2a155fa1
442e4dc68ce5049cbe3f8c3a04ae8998c13992565e76e6fa4ce58d628991301
f0c286c96c0b841c3c6c4337cfd0261de9c7ae990b605a1d272ffcb0e7d8a032
71
```

El cifrado cambia porque RSA-OAEP usa aleatoriedad para mayor seguridad.

Ejercicio 12

En este ejercicio nos debemos comunicar con una empresa y se nos indica que tenemos que usar AES/GCM en la comunicación y se afirma que nuestro sistema utilizara **misma clave** y **mismo nonce**, **en cada** comunicación con el tercero

A continuación, se nos presentan los siguientes datos para realizar el cifrado, que repetimos, salvando el mensaje, el sistema de esta empresa está **reutilizando** la **clave** y el **nonce** de forma constante

Mensaje: “He descubierto el error y no volveré a hacerlo mal”

Key:

E2CFF885901B3449E9C448BA5B948A8C4EE322152B3F1ACFA0148FB3A4
2 6DB74

Nonce: 9Yccn/f5nJJhAt2S

Se nos indica también que el texto tiene que estar presentado en **hexadecimal** y **base64**

```
EJERCICIO 12.py X
C: > Users > Programacion Juan > Desktop > EJERCICIO 12 > EJERCICIO 12.py > ...

1
2  #EJERCICIO 12 - PRACTICA FINAL - CRIPTOGRAFIA
3
4  from Crypto.Cipher import AES
5  import base64
6
7  # Texto a cifrar
8  texto_a_cifrar = "He descubierto el error y no volveré a hacerlo mal".encode("utf-8")
9
10 # Clave en hexadecimal
11 clave_hex = "E2CFF885901B3449E9C448BA5B948A8C4EE322152B3F1ACFA0148FB3A426DB74"
12 clave_bytes = bytes.fromhex(clave_hex)
13
14 # Nonce en Base64
15 nonce_base64 = "9Yccn/f5nJJhAt2S"
16 nonce_bytes = base64.b64decode(nonce_base64)
17
18 # AES-GCM (cifrado)
19 cifrado = AES.new(clave_bytes, AES.MODE_GCM, nonce=nonce_bytes) #PREPARAMOS EL CIFRADOR
20
21 texto_cifrado, tag = cifrado.encrypt_and_digest(texto_a_cifrar) #CIFRAMOS + MAC (TAG DE AUTENTICACION)
22
23 # Salida en Hexadecimal y Base64
24 print("Texto cifrado (hex):", texto_cifrado.hex())
25 print("Texto cifrado (base64):", base64.b64encode(texto_cifrado).decode())
26
27 # MOSTRAMOS EL MAC/TAG
28 print("MAC / TAG (hex):", tag.hex())
29 print("MAC / TAG (base64):", base64.b64encode(tag).decode())
30
31
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Juan\AppData\Local\Python\p
Texto cifrado (hex): 5dcb6261d0fba29ce39431e9a013b34cbca2a4e04bb2d90149d61f4afd04d65e2abdd9d84bba6eb8307095f5078fbfc16256d
Texto cifrado (base64): Xcu2Jh0PuIn00UMemgE7NMvKKk4Euy2QFJ1h9K/QTWXiq92dhLum64MHCv9QePv8Fivt
MAC / TAG (hex): 6120e37aa4c3ecfd9261640dcc46410d
MAC / TAG (base64): YSDjeqTD7P2SYWQNZEBDQ==
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> |
```

Para resolver esta consigna se implementó un script en Python que permite cifrar el mensaje utilizando AES-GCM, donde se separó con “#” cada sección.

Primero que nada, importamos el algoritmo **AES** y el módulo de **base64**, ya que el nonce está en base64, y a diferencia de **hexadecimal**, este no viene incorporado. Luego ingresamos todos los datos de entrada primarios que se nos dieron (Mensaje, Clave y Nonce) y hacemos que sean interpretados como bytes.

Una vez tenemos todos los datos, preparamos el cifrador agregando **AES** y el modo **GCM** y por último ciframos y generamos el **tag**, para que se cree el **MAC** o **TAG** de Autenticación

Finalmente, se agregó la salida tanto en formato hexadecimal como en Base64, tal como lo indica la consigna. Asimismo, se muestra en la terminal el MAC/tag generado, también presentado en ambos formatos.

NO OBSTANTE, la consigna nos plantea una pregunta que es de suma importancia:

“ ¿Qué estamos haciendo mal? “

Respondiendo a esta pregunta, lo que sucede es que si bien es común a veces usar la misma clave, jamás deberíamos reutilizar el **Nonce**, ya que al hacerlo se podría comprometer la seguridad del cifrado. Deja abiertas puertas que permiten que se pueda obtener información de los mensajes.

Una solución para este problema sería generar un nonce nuevo en cada comunicación, usando valores aleatorios. De esta manera no se reutiliza el mismo nonce con la misma clave, evitando que se comprometa la seguridad del cifrado

Para esto al inicio del código en conjunto a importar AES y base64, agregamos también la opción de obtener bytes aleatorios. Esto nos quedaría así:

```
# EJERCICIO 12 - PRACTICA FINAL - CRIPTOGRAFIA

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
import base64
```

Luego en la línea del NONCE propiamente dicha, suplantamos lo que teníamos por lo siguiente:

```
15 # Nonce generado de forma aleatoria (NO SE REPITE)
16 nonce_bytes = get_random_bytes(12)
```

Por último, adherimos la opción para que se nos muestre el **nonce** en la terminal también en ambos formatos

```
32 # MOSTRAMOS EL NONCE UTILIZADO
33 print("Nonce (hex):", nonce_bytes.hex())
34 print("Nonce (base64):", base64.b64encode(nonce_bytes).decode())
```

Resultado en Terminal (con random nonce):

Texto cifrado (hex):

f820767eee1ea9d68041eee171416c48b9173fb02667871f112e92c4f64ce187
de151dae62c66568ab8b816f96d51faf469661

Texto cifrado (base64):

+CB2fu4eqdaAQe7hcUFsSLkXP7AmZ4cfES6SxPZM4YfeFR2uYsZlaKuLgW
+W1R+vRpZh

MAC / TAG (hex): 302d4328b692be035a8f48a7ec91fe60

MAC / TAG (base64): MC1DKLaSvgNaj0in7JH+YA==

Nonce (hex): bf5d337fcbda0d332cd75ba9

Nonce (base64): v10zf8vaDTMs11up

Ejercicio 13

Aquí se nos pide realizar una Firma Digital con el algoritmo PKCS#1 v1.5, utilizando las claves contenidas en los ficheros **.pem** que utilizamos en el **Ejercicio 11**, para el siguiente mensaje:

“El equipo está preparado para seguir con el proceso, necesitaremos más recursos.”

Se nos pide calcular la firma (en hexadecimal) con la curva elíptica **ed25519**, usando las claves **ed25519priv** y **ed25519-publ**

Al analizar la consigna se observa que se piden dos firmas digitales distintas sobre el mismo mensaje, una utilizando **RSA** con **PKCS#1 v1.5** y otra utilizando **Ed25519** (curva elíptica). Por este motivo, se decidió separar la

resolución del ejercicio en dos partes, implementando cada firma de forma independiente y en códigos separados.

Parte 1 - Firma RSA (PKCS#1 v1.5)

```
EJERCICIO 13_FIRMA_RSA PKCS#1 v1.5.py 3 X
C: > Users > Programacion Juan > Desktop > EJERCICIO 13 > EJERCICIO 13_FIRMA_RSA PKCS#1 v1.5.py > ...
1 # EJERCICIO 13 - PRACTICA FINAL - CRIPTOGRAFIA - FIRMA DIGITAL CON RSA PKCS#1 v1.5
2
3 import os
4 from Crypto.PublicKey import RSA
5 from Crypto.Hash import SHA256
6 from Crypto.Signature import pkcs1_15
7
8 # 1) MENSAJE A FIRMAR
9 mensaje = "El equipo está preparado para seguir con el proceso, necesitaremos más recursos."
10 mensaje_bytes = mensaje.encode("utf-8")
11
12 # 2) ASIGNAMOS RUTA BASE (LA MISMA CARPETA DONDE ESTA ESTE SCRIPT)
13 BASE_DIR = os.path.dirname(__file__)
14
15 # 3) CARGAMOS LA CLAVE PRIVADA .PEM
16 path_rsa_priv = os.path.join(BASE_DIR, "clave-rsa-oaep-priv.pem") # UBICADA EN LA MISMA CARPETA QUE EL SCRIPT
17 clave_privada_rsa = RSA.import_key(open(path_rsa_priv, "rb").read())
18
19 # 4) HASHEAMOS EL MENSAJE
20 hash_mensaje = SHA256.new(mensaje_bytes)
21
22 # 5) GENERAMOS LA FIRMA RSA
23 firma_rsa = pkcs1_15.new(clave_privada_rsa).sign(hash_mensaje)
24
25 # 6) MOSTRAMOS LA FIRMA EN HEXADECIMAL
26 print("Firma RSA (hex):", firma_rsa.hex())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v ... | C X

PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Juan\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\Programacion Juan\Desktop\EJERCICIO 13\EJERCICIO 13_FIRMA_RSA PKCS#1 v1.5.py"

Firma RSA (hex): e4506c518e0e2b443255a3626f3f23b77b9d5e1e4d6b3dcf90f7e118d6063950a23885c6dece92aa3d6eff2a72886b2552be969e11a4b7441bdeadc596c1b94e67a8f941ea998ef08b2cb3a925c959bcaae2ca9e6e0f95b989c709b9a0b90a0c69d9eaccd863bc924e70450ebb87369d721a9ec798fe6308e045417d0a56b86d84b305c55a0e766190d1ad0934a1befbbe031853277569f8383846d971d0daf05d023545d274f1bdd4b00e8954ba39dacc4a0875208f36d3c9207af096ea0f0d3baa752b48545a5d79cce0c2ebb6ff601d92978a33c1a8a707c1ae1470a09663ac6b9519391b61891bf5e066999a0a0dbae21f0aaaa6f9b9d59f41928d

PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> |

En esta primera parte del ejercicio se realiza una **firma digital** utilizando **RSA** con **PKCS#1 v1.5**. Para esto se reutiliza la clave privada **RSA** en formato **.pem** que ya habíamos usado en el Ejercicio 11.

Comenzamos tomando el mensaje en la consigna y pasándolo a **bytes**, ya que Python trabaja directamente con este formato. Luego se importa la clave privada **RSA** desde el archivo correspondiente, ubicado en la misma carpeta que el **script**, de la misma manera que se hizo en ejercicios anteriores.

Una vez cargada la **clave**, se aplica el algoritmo **SHA-256** al mensaje para obtener su **hash**.

Con el hash generado y la clave privada, se procede a crear la firma digital utilizando **PKCS#1 v1.5**, y finalmente se muestra el resultado en hexadecimal, tal como lo pide la consigna.

Resultado:

Firma RSA (hex):

a4606c518e0e2b443255e3626f3f23b77b9d5e1e4d6b3dcf90f7e118d606395
0a23885c6dece92aa3d6eff2a72886b2552be969e11a4b7441bdeadc596c1b9
4e67a8f941ea998ef08b2cb3a925c959bcaae2ca9e6e60f95b989c709b9a0b9
0a0c69d9eaccd863bc924e70450ebbbb87369d721a9ec798fe66308e045417
d0a56b86d84b305c555a0e766190d1ad0934a1befbbe031853277569f83838
46d971d0daf05d023545d274f1bdd4b00e8954ba39dacc4a0875208f36d3c92
07af096ea0f0d3baa752b48545a5d79cce0c2ebb6ff601d92978a33c1a8a707c
1ae1470a09663acb6b9519391b61891bf5e06699aa0a0dbae21f0aaaa6f9b9d
59f41928d

Parte 2 - Firma Curva Elíptica (ED25519)

```
EJERCICIO 13_FIRMA_ED25519_C_ELIPTICA.py 2 X
C: > Users > Programacion Juan > Desktop > EJERCICIO 13 > EJERCICIO 13_FIRMA_ED25519_C_ELIPTICA.py > ...
1 # EJERCICIO 13 - PRACTICA FINAL - CRIPTOGRAFIA - FIRMA DIGITAL CON CURVA ELIPTICA ED25519
2
3 from Crypto.PublicKey import ECC
4 from Crypto.Signature import eddsa
5
6 # 1) MENSAJE A FIRMAR
7 mensaje = "El equipo está preparado para seguir con el proceso, necesitaremos más recursos."
8 mensaje_bytes = mensaje.encode("utf-8")
9
10 # 2) CLAVE PRIVADA ED25519 EN HEX (CONVERTIDA EN CYBERCHEF)
11 clave_privada_hex = "c4225628eb2fc184e90bcadec9bf6d8596f372473b1bb9fb62cc975aacf34cbc7cbd8471d56e8aef76809d025e19df3b17f7127e6d440f91c1f4
12
13 # 3) PASAMOS DE HEX A BYTES
14 clave_bytes = bytes.fromhex(clave_privada_hex)
15
16 # 4) CONSTRUIMOS LA CLAVE PRIVADA ED25519 (USANDO SEED, NO d)
17 clave_privada_ed = ECC.construct(
18     curve="Ed25519",
19     seed=clave_bytes[:32]
20 )
21
22 # 5) FIRMAMOS
23 firmador = eddsa.new(clave_privada_ed, mode="rfc8032")
24 firma_ed = firmador.sign(mensaje_bytes)
25
26 # 6) MOSTRAMOS LA FIRMA EN HEXADECIMAL
27 print("Firma Ed25519 (hex):", firma_ed.hex())
28
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Juan\AppData\Local\Python\pythoncore-3.14-64\python.exe"
ED25519_C_ELIPTICA.py"
Firma Ed25519 (hex): bf32592dc235a26e31e231063a1984bb75ffd9dc5550cf30105911ca4560dab52abb40e4f7e2d3af828abac1467d95d668a80395e0a71c51798bd54469b7360d
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code>
```

En esta parte del ejercicio se genera una **firma digital** utilizando el algoritmo de Curva Elíptica **ED25519**. Para ello, se toma el mensaje indicado en la consigna y se pasa a **bytes**, ya que es el formato requerido para realizar la firma.

Luego se utiliza la clave privada “**ed25519-priv**”, ubicada en el repositorio de la práctica, la cual por cuestiones de errores me vi obligado a convertir previamente a formato hexadecimal, desde la web de **cyberchef**. Luego se pegó en el apartado correspondiente y se transformó a bytes para poder ser utilizada dentro del script. A partir de esta clave se construye la clave privada correspondiente y se procede a firmar el mensaje.

Finalmente, se muestra el valor de la firma digital obtenida en formato hexadecimal, tal como lo solicita el enunciado.

Resultado:

Firma Ed25519 (hex):

bf32592dc235a26e31e231063a1984bb75ffd9dc5550cf30105911ca4560dab52abb40e4f7e2d3af828abac1467d95d668a80395e0a71c51798bd54469b7360d

Ejercicio 14

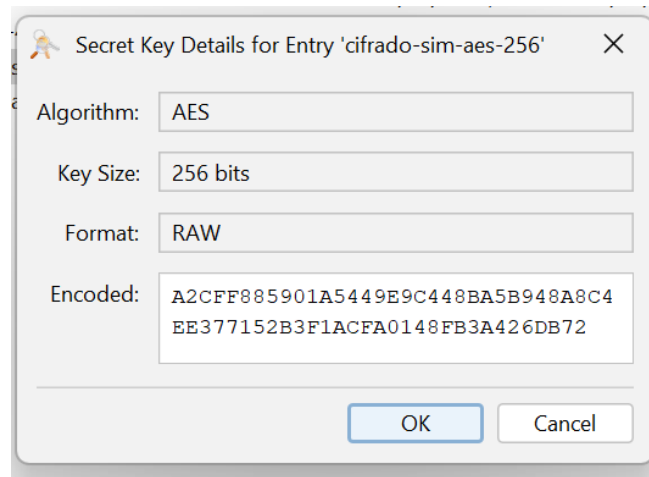
En este penúltimo ejercicio se nos presenta un caso donde debemos de generar una nueva clave **AES**, utilizando una **HKDF** con un hash **SHA-512**. Para esto debemos de utilizar la clave maestra que se encuentra dentro de la keystore de la práctica, bajo el nombre de “**cifrado-sim-aes-256**”. La clave obtenida dependerá de un identificador de dispositivo, en este caso tendrá el valor en hexadecimal:

e43bb4067cbcfab3bec54437b84bef4623e345682d89de9948fbb0afedc461a3



Bajando esto a tierra entonces podemos decir que básicamente partimos de una clave **AES** guardada en la keystore y, utilizando como **SALT** el identificador de dispositivo y **HKDF** con **SHA-512**, derivamos una **nueva clave**.

En primera instancia el único dato que nos falta tener claro para empezar la construcción de nuestro archivo Python es la clave maestra dentro de la **keystore** por lo que ingresaremos a ella y extraemos la **clave maestra**:



Una vez tenemos este ultimo dato faltante procedemos a construir el ejercicio dentro de Python:

```
EJERCICIO 14_Derivacion_de_Clave.py X
C: > Users > Programacion Juan > Desktop > EJERCICIO 14 > EJERCICIO 14_Derivacion_de_Clave.py > ...
1  # EJERCICIO 14 - PRACTICA FINAL - CRIPTOGRAFIA -DERIVACION DE CLAVE CON HKDF + SHA-512
2
3  from Crypto.Protocol.KDF import HKDF
4  from Crypto.Hash import SHA512
5
6  # 1) CLAVE MAESTRA AES (KEYSTORE)
7  clave_maestra_hex = "A2CFF885901A5449E9C448BA5B948A8C4EE377152B3F1ACFA0148FB3A426DB72"
8  clave_maestra_bytes = bytes.fromhex(clave_maestra_hex)
9
10 # 2) IDENTIFICADOR DE DISPOSITIVO (SALT) EN HEX
11 salt_hex = "e43bb4067cbcfab3bec54437b84bef4623e345682d89de9948fbb0afedc461a3"
12 salt_bytes = bytes.fromhex(salt_hex)
13
14 # 3) DERIVAMOS UNA NUEVA CLAVE AES DE 256 BITS
15 clave_derivada = HKDF(
16     master=clave_maestra_bytes,
17     key_len=32,
18     salt=salt_bytes,
19     hashmod=SHA512
20 )
21
22 # 4) MOSTRAMOS LA CLAVE DERIVADA
23 print("Clave AES derivada (hex):", clave_derivada.hex())
24
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Ju
vacion_de_Clave.py"
Clave AES derivada (hex): e716754c67614c53bd9bab176022c952a08e56f07744d6c9edb8c934f52e448a
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Ju
vacion_de_Clave.py"
Clave AES derivada (hex): e716754c67614c53bd9bab176022c952a08e56f07744d6c9edb8c934f52e448a
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code>
```

Primero se ingresa la **clave maestra** que se encuentra almacenada en la **keystore**, la cual está en formato **hexadecimal** y se convierte a **bytes** para poder utilizarla dentro del script.

Luego se toma el **identificador de dispositivo** dado por la consigna, también en hexadecimal, el cual se usa como **SALT** dentro del proceso de derivación. Con estos dos valores se aplica el algoritmo **HKDF** utilizando **SHA-512**, obteniendo como resultado una nueva clave **AES** de **256 bits**.

Por último, la clave derivada se muestra en formato hexadecimal, tal como se pide en la consigna.

Resultado en Terminal:

Clave AES derivada (hex):

e716754c67614c53bd9bab176022c952a08e56f07744d6c9edb8c934f52e448a

EJERCICIO 15

```
EJERCICIO 15_KEYBLOCK.py 1 X
C: > Users > Programacion Juan > Desktop > EJERCICIO 15 > EJERCICIO 15_KEYBLOCK.py > ...
1  # EJERCICIO 15 - PRACTICA FINAL - CRIPTOGRAFIA - IMPORTACION Y DESENVOLVIMIENTO DE UN KEY BLOCK TR-31
2
3  from psec import tr31
4
5  # Clave de transporte (KBPK) en hexadecimal
6  kbpk_hex = "A1A10101010101010101010101010102"
7  kbpk_bytes = bytes.fromhex(kbpk_hex)
8
9  # Key Block TR-31 recibido
10 keyblock_tr31 = "D0144D0A00S000042766B926582DF93AE6E29B58135B77A2F616C8D515ACDBE6A5626F79FA7B4071E9EE1423C6D7970FA2B965D18B23922B5B2E5657495E03CD857FD37018E111B"
11
12 # Creamos el KeyBlock con la clave de transporte
13 kb = tr31.KeyBlock(kbpk_bytes)
14
15 # Cargamos la cabecera del bloque para poder leer su información
16 kb.header.load(keyblock_tr31)
17
18 # Desenvolvemos (unwrap) el Key Block para obtener la clave real
19 clave_importada = kb.unwrap(keyblock_tr31)
20
21 # Mostramos resultados
22 print("Clave importada (hex):", clave_importada.hex().upper())
23 print("Version:", kb.header.version_id)
24 print("Uso de clave:", kb.header.key_usage)
25 print("Algoritmo:", kb.header.algorithm)
26 print("Modo de uso:", kb.header.mode_of_use)
27 print("Exportabilidad:", kb.header.exportability)
28
29
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Programacion Juan\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:/Users/Programacion OCK.py"
Clave importada (hex): C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1
Version: D
Uso de clave: D0
Algoritmo: A
Modo de uso: B
Exportabilidad: S
PS C:\Users\Programacion Juan\AppData\Local\Programs\Microsoft VS Code> |
```

Para resolver esta consigna armé un script en Python que permite importar y trabajar con un **Key Block** usando el estándar **TR-31**.

Primero se ingresa la **clave de transporte (KBPK)**, que está en formato hexadecimal. Esta clave se pasa a bytes para poder usarla dentro del script y es la que se utiliza para poder **desenvolver (unwrap)** el Key Block que nos envían.

Después se carga el **Key Block TR-31** que nos da la consigna. A partir de ese bloque se lee la **cabecera**, lo que nos permite ver información básica de la clave, como la versión del bloque, el uso de la clave, el algoritmo, el modo de uso y si la clave es exportable o no.

Una vez leída la cabecera, se realiza el proceso de unwrap, que básicamente consiste en extraer la clave real que está protegida dentro del Key Block. Esta clave se obtiene solo en memoria y se muestra en formato hexadecimal junto con los datos descriptivos del bloque.

Para poder ejecutar este script fue necesario instalar una librería externa llamada psec, ya que Python por defecto no tiene soporte para trabajar con el estándar TR-31. Esta librería es la que permite interpretar correctamente el Key Block y realizar el proceso de importación de la clave.

Con este mecanismo se garantiza que la clave nunca viaje en claro, ya que siempre está protegida dentro del bloque **TR-31** y solo puede ser recuperada por quien tenga la clave de transporte correspondiente.

Resultado en Terminal:

Clave importada (hex): C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1C1

Versión: D

Uso de clave: D0

Algoritmo: A

Modo de uso: B

Exportabilidad: S